High Level Design
on
Sports Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Pranjali Sharma	EN22CS301733
2	Prithviraj Singh Tomar	EN22CS301757
3	Priyanshi Gandhe	EN22CS301764
4	Priyanshu Sengar	EN22CS301770
5	Rudra Yadav	EN22CS301831

Group Name: 07D7

Project Number: AAI-30

Industry Mentor Name:

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year: 2026

Table of Contents

1. Introduction
   1.1 Scope of the document
   1.2 Intended Audience
   1.3 System overview

2. System Design
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The AI Sports Content Generator is a web-based application that leverages Generative AI and real-time sports data to automatically create high-quality sports articles. The system integrates live cricket match data with a large language model to generate content that mimics professional sports journalism platforms.

The application provides user authentication, article generation, history tracking, and a trending live matches section, offering a complete AI-powered sports content experience.

1.1 Scope of the Document

This document describes the High-Level Design (HLD) of the system, including:

- Overall architecture
- Key modules and their interactions
- Data flow and processing
- API integrations
- Design considerations
- Non-functional requirements

It serves as a reference for developers, reviewers, and stakeholders to understand the system design.

1.2 Intended Audience

This document is intended for:

- Software Developers
- System Architects
- Project Evaluators / Interviewers
- QA/Test Engineers

It assumes familiarity with web applications, APIs, and basic system design concepts.

1.3 System Overview

The system is a single-page web application built using Streamlit that integrates:

- Frontend + Backend Logic: Streamlit
- AI Engine: Groq API (LLaMA 3 model)
- Live Data Source: CricAPI
- Database: SQLite
- Authentication: bcrypt-based password hashing

Key Features:

- User Login & Registration
- AI-generated sports articles
- Real-time match data integration
- Article history storage
- Trending live matches panel (IPL 2026)

2. System Design

2.1 Application Design

The application follows a layered architecture:

Presentation Layer (Streamlit UI)

↓

Application Logic Layer

↓

Service Layer (AI + Match API)

↓

Data Layer (SQLite)

Layers:

1. Presentation Layer o Handles UI rendering and user interaction o Includes sidebar navigation and main content
2. Application Layer o Controls flow (login, page switching) o Manages session state
3. Service Layer o Fetches live match data o Generates AI articles
4. Data Layer o Stores user credentials and article history

2.2 Process Flow

<u>Article Generation Flow:</u>

User Login → Enter Topic → Click Generate

↓

Fetch Live Match Data (CricAPI)

↓

Create Prompt (Query + Match Data)

↓

Call Groq API

↓

Receive AI Article

↓

Display Output + Save in DB

History Flow:

User clicks History

↓

Fetch records from SQLite

↓

Display using expandable UI

2.3 Information Flow

User Input → Streamlit UI → Controller Logic

↓

Match Data API → Process → Combine with Query

↓

Groq LLM → Generate Article

↓

Return Response → Display + Store in DB

2.4 Components Design 1.

UI Component

- Sidebar navigation
- Article input form
- Trending matches column

2. Authentication Component

- Register user
- Login validation

3. Match Service Component

- Fetch and parse CricAPI data

4. AI Service Component

- Construct prompt
- Call Groq API

5. History Component

- Save articles
- Retrieve user history

2.5 Key Design Considerations

- Simplicity: Single-file architecture for easy deployment
- Real-time Data: Uses API instead of static dataset
- Modularity: Separate functions for each service
- User Experience: Clean UI with sidebar and trending section
- Performance: Minimal DB operations and lightweight storage

2.6 API Catalogue

1. CricAPI (External API)

- Endpoint:

https://api.cricapi.com/v1/currentMatches

- Purpose:
  Fetch live cricket match data

2. Groq API (LLM)

- Model:

  llama-3.3-70b-versatile

- Purpose:
  Generate sports articles

3. Internal APIs (Functions)

Function	Purpose
get_match_data()	Fetch match data
generate_article()	Generate AI content
register_user()	Register new user
login_user()	Authenticate user
save_history()	Store article
load_history()	Retrieve articles

3. Data Design

3.1 Data Model

Users Table username TEXT

PRIMARY KEY password TEXT

History Table

id INTEGER PRIMARY KEY AUTOINCREMENT

username TEXT query TEXT article TEXT

timestamp TEXT

3.2 Data Access Mechanism

- Uses SQLite with direct SQL queries
- Connection maintained using sqlite3
- Queries executed via cursor

3.3 Data Retention Policies

- Articles are stored permanently unless deleted manually
- No auto-cleanup mechanism implemented
- Can be extended with retention limits

3.4 Data Migration

- Currently not required due to SQLite
- Future migration:

- SQLite → PostgreSQL/MySQL
- Export via CSV or SQL dump

4. Interfaces

User Interface:

- Web-based UI using Streamlit

External Interfaces:

- CricAPI for live data

Groq API for AI content generation Internal Interfaces:

- Function calls between modules

5. State and Session Management

- Managed using st.session_state

Key Variables:

- user → current logged-in user
- page → current active page

Behavior:

- Maintains session across interactions
- Handles navigation and authentication state

6. Caching

Currently minimal caching is used.

Possible Improvements:

- Cache match data for short duration
- Avoid repeated API calls

Example:

@st.cache_data def

get_match_data():

7. Non-Functional Requirements

7.1 Security Aspects

- Password hashing using bcrypt
- API keys stored in .env
- No plaintext password storage
- Input validation for queries

7.2 Performance Aspects

- Fast UI rendering with Streamlit
- Groq provides low-latency inference
- SQLite ensures quick read/write for small datasets

Bottlenecks:

- API latency (CricAPI, Groq)
- Network dependency

8. References

- Streamlit Documentation
- Groq API Documentation
- CricAPI Documentation
- SQLite Documentation
- bcrypt Library Documentation